Part 2

Functional design and combinator libraries

We said in chapter 1 that functional programming is a radical premise that affects how we write and organize programs at every level. In part 1, we covered the fundamentals of FP and saw how the commitment to using only pure functions affects the basic building blocks of programs: loops, data structures, exceptions, and so on. In part 2, we'll see how the assumptions of functional programming affect *library design*.

We'll create three useful libraries in part 2—one for parallel and asynchronous computation, another for testing programs, and a third for parsing text. There won't be much in the way of new syntax or language features, but we'll make heavy use of the material already covered. Our primary goal isn't to teach you about parallelism, testing, and parsing. The primary goal is *to help you develop skill in designing functional libraries*, even for domains that look nothing like the ones here.

This part of the book will be a somewhat meandering journey. Functional design can be a messy, iterative process. We hope to show at least a stylized view of how functional design proceeds in the real world. Don't worry if you don't follow every bit of discussion. These chapters should be like peering over the shoulder of someone as they think through possible designs. And because no two people approach this process the same way, the particular path we walk in each case might not strike you as the most natural one—perhaps it considers issues in what seems like an odd order, skips too fast, or goes too slow. Keep in mind that when you design your own functional libraries, you get to do it at your own pace,

take whatever path you want, and, whenever questions come up about design choices, you get to think through the consequences in whatever way makes sense for you, which could include running little experiments, creating prototypes, and so on.

There are no right answers in functional library design. Instead, we have a collection of *design choices*, each with different trade-offs. Our goal is that you start to understand these trade-offs and what different choices mean. Sometimes, when designing a library, we'll come to a fork in the road. In this text we may, for pedagogical purposes, deliberately make a choice with undesirable consequences that we'll uncover later. We want you to see this process first-hand, because it's part of what actually occurs when designing functional programs. We're less interested in the particular libraries covered here in part 2, and more interested in giving you insight into how functional design proceeds and how to navigate situations that you will likely encounter. Library design is not something that only a select few people get to do; it's part of the day-to-day work of ordinary functional programming. In these chapters and beyond, you should absolutely feel free to experiment, play with different design choices, and develop your own aesthetic.

One final note: as you work through part 2, you may notice repeated patterns of similar-looking code. Keep this in the back of your mind. When we get to part 3, we'll discuss how to remove this duplication, and we'll discover an entire world of fundamental abstractions that are common to *all libraries*.